ResQFlow

ML-Based Dynamic Emergency Vehicle Routing System

Project Technical Report
Foundation of Data Science - BCSE206L

Submitted By
Name: Joyster Pinto
Reg No: 24BCE1245
Slot: B2
GitHub Repo: https://github.com/Joyster-pinto/ResQFlow_ML

Table of Contents

1. Abstract 3

2. Introduction & Problem Statement 3

3. SUMO Simulation Environment 4

3.1 Eclipse SUMO Platform 4

3.2 Mysore Network Topology 5

3.3 Road Type Classification 5

3.4 Dynamic Traffic Scaling via TraCI 6

4. Data Engineering & Synthetic Dataset Generation 6

4.1 Rationale for Synthetic Data 6

4.2 Generation Methodology 6

4.3 Dataset Specifications 7

4.4 Feature Correlation Analysis 8

5. Machine Learning Architecture 9

5.1 Design Philosophy: Ensemble Over Single-Model 9

5.2 Component Model Details 9

5.3 Ensemble Prediction 10

6. Predictive Feature Set 10

7. Dynamic Scenario Modifiers 11

7.1 Predicted Path Visualisations 11

8. Performance Validation 14

8.1 Test Scenario 14

8.2 Exact Travel Time Results — K.R. Hospital to Vijayanagar 14

8.3 Travel Time Comparison Chart 15

8.4 Key Observations 15

9. System Architecture Summary 16

10. Conclusion 17

1. Abstract

ResQFlow is an AI-driven emergency vehicle routing system designed to address a fundamental shortcoming of conventional GPS-based navigation: the reliance on static speed limits and geometric distances. Standard shortest-path algorithms fail under the dynamic, unpredictable conditions characteristic of dense urban emergencies — monsoon weather, traffic accidents, flash floods, and peak-hour gridlocks. ResQFlow counters this by deploying a three-model hybrid Machine Learning ensemble combining LightGBM, CatBoost, and a Graph Neural Network (GraphSAGE) to predict real-time traversal costs across the Mysore road network. The system dynamically identifies and avoids high-congestion corridors, routing emergency vehicles via physically clearer paths that minimise actual travel time rather than geometric distance. The platform is validated through Eclipse SUMO (Simulation of Urban MObility), using the full-fidelity Mysore city network of 10,696 junctions and 30,631 raw edges, with the ML routing engine operating on a filtered active graph of 17,586 passenger-accessible road segments and dynamic vehicle populations scaling up to 1,300+ concurrent agents.

2. Introduction & Problem Statement

Emergency response time is one of the most critical determinants of outcomes in life-threatening situations. Every additional minute of travel time for an ambulance, fire truck, or police unit carries a measurable cost. Despite this, modern emergency dispatch systems predominantly rely on the same consumer-grade navigation stack as regular commuter traffic — algorithms that optimise for static geometric distance under assumed free-flow speed conditions.

This approach has two fundamental failure modes in urban environments:

- Static road cost models assume a vehicle travels at or near the legal speed limit, ignoring live congestion, intersection queues, road hazards, and weather degradation.
- Shortest-path algorithms (e.g., Dijkstra) have no mechanism to anticipate or route around developing bottlenecks — they treat all roads as equally traversable at any given moment.

ResQFlow is engineered specifically to address both failure modes. The system integrates a real-time traffic feature vector for every road segment into a predictive ML model that outputs accurate traversal time estimates, then feeds those estimates into a modified shortest-path router. The result is a system capable of dynamically diverting emergency routes around congestion, hazards, or adverse weather in real time.

3. SUMO Simulation Environment

3.1 Eclipse SUMO Platform

Eclipse SUMO (Simulation of Urban MObility) is an open-source, microscopic, multi-modal traffic simulation suite. It enables deterministic, physically realistic simulation of individual vehicle agents moving through a road network defined by precise geometric and regulatory constraints. ResQFlow uses SUMO as both the data generation environment (synthetic training data) and the live validation platform for ML-guided routing.

The Python application interfaces with SUMO via the TraCI (Traffic Control Interface) API, enabling bidirectional real-time interaction with the running simulation — reading live vehicle states, edge occupancies, and injecting dynamic routing commands for the simulated emergency vehicle.

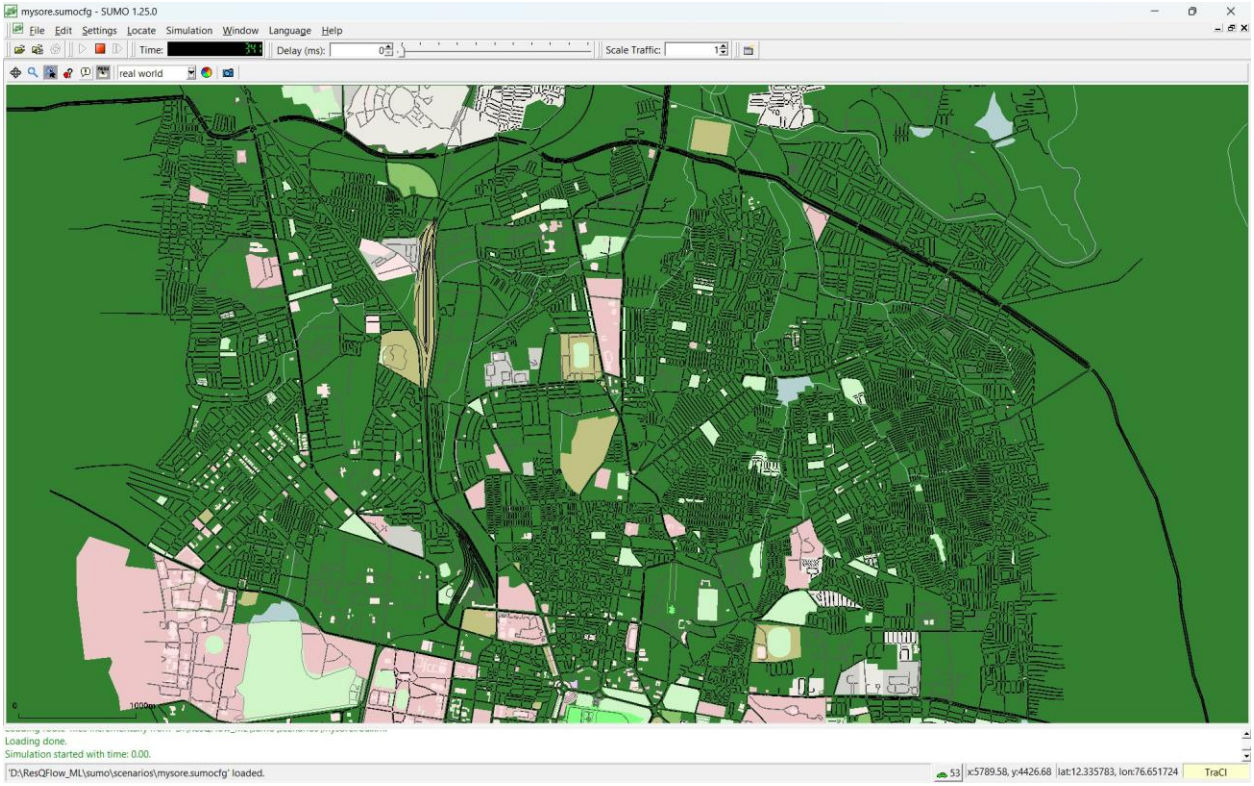

Figure 1: Eclipse SUMO 1.25.0 — Full Mysore city network loaded (mysore.sumocfg). The dense road mesh represents the 10,696-junction, 30,631-edge urban network with TraCI interface active.

3.2 Mysore Network Topology

The entire physical road geometry of Mysore, Karnataka, has been embedded natively into the SUMO network file (mysore.net.xml). This includes precise road geometry, intersection logic, traffic signal timing, lane configurations, speed limits, and road classifications. The network statistics below reflect values extracted directly from the source network file and PyTorch Geometric graph construction:

Network Component	Count	Notes
Total Graph Nodes (Junctions)	**10,696**	All intersections, endpoints & merge points in mysore.net.xml
Total Raw Edges	**30,631**	Includes pedestrian paths, bike lanes & internal intersection geometry
Active ML Graph Edges	**17,586**	Passenger-accessible roads > 20m; the live graph the AI routes across
Road Type Classes	**4 (codes 0–3)**	Residential, Local, Arterial, Highway — encoded via CatBoost road_type_code
Traffic Signal Junctions	**Present**	Native SUMO traffic light phases embedded in network
Baseline Vehicle Fleet	**721 agents**	Continuously circulating (mysore.rou.xml)
Rush Hour Vehicle Fleet	**1,300+ agents**	1.8× scale multiplier injected via TraCI at runtime
Network Source File	**mysore.net.xml**	Full geometry, speed limits, intersections
Route Definition File	**mysore.rou.xml**	Vehicle trips and routing parameters

3.3 Road Type Classification

Rather than using text labels which add processing overhead, CatBoost's road_type_code feature classifies all 17,586 active edges into four numeric classes derived directly from their physical speed limits:

Code	Road Class	Speed Limit Range	Typical Examples
0	Residential	≤ 30 km/h	Side streets, colony roads, narrow lanes
1	Local	31 – 50 km/h	Inner-city roads, market streets
2	Arterial	51 – 70 km/h	Major inner-city avenues, main roads
3	Highway	> 70 km/h	Ring road around Mysore, expressways

3.4 Dynamic Traffic Scaling via TraCI

The simulation is not static. During a live software sequence, the Python application bridges to SUMO via TraCI and actively modulates traffic density in real time. A Rush Hour profile runs the simulation with a 1.8× scale multiplier, programmatically injecting additional vehicle agents into the network to physically stress-test the ML routing engine within the SUMO GUI. This allows validation of routing decisions against a fully simulated, congested urban environment.

4. Data Engineering & Synthetic Dataset Generation

4.1 Rationale for Synthetic Data

High-resolution, real-time historical traffic data at the individual road-segment level for Mysore is not available in the public domain. Rather than compromising the model by training on low-fidelity or geographically mismatched data, ResQFlow employs a simulation-driven synthetic data generation pipeline rooted in established traffic flow physics. This approach yields a dataset that is both mathematically rigorous and contextually accurate to the Mysore road network.

4.2 Generation Methodology

- Network Extraction: Road typologies, speed limits, edge lengths, lane counts, and intersection constraints are extracted directly from mysore.net.xml.
- Physics-Based Simulation: Traffic behaviour is modelled using established relationships — peak-hour periods drive exponential increases in vehicle volume, and higher vehicle density mathematically degrades mean traversal speed.
- Temporal Variation: Time-of-day patterns are incorporated, distinguishing morning peak (07:00–09:00), evening peak (17:00–19:00), off-peak, and overnight conditions.
- Scenario Injection: Adverse event modifiers (accidents, construction, flooding) are injected stochastically to ensure the model trains on realistic edge cases.

4.3 Dataset Specifications

The hybrid architecture of ResQFlow necessitates two fundamentally different training datasets, one for each model tier:

Dataset 1 — Tabular CSV (LightGBM & CatBoost)

Because highly granular, edge-by-edge real-time historical traffic data for Mysore does not exist publicly, a custom synthetically engineered CSV dataset is used. It mathematically models accurate traffic physics — including exponential congestion curves, peak-hour load spikes, and geometric bottleneck tracking — derived directly from the properties of the Mysore SUMO network. The generator produces 10,000 records, then aggressively drops the top 1% statistical outliers to prevent skewed training, resulting in exactly 9,900 clean training samples.

Dataset Type	Synthetic CSV (physics-based simulation)
Training Samples	9,900 records (10,000 generated, top 1% outliers removed)
Target Variable	travel_time — actual traversal time (seconds) under current conditions
Feature Count	13 input features (traffic physics, geometry, temporal, weather)
Excluded Features	Latitude, Longitude, Street Names — to prevent geographic overfitting
Used By	LightGBM Regressor + CatBoost Regressor

Dataset 2 — PyTorch Geometric Spatial Graph (RoadGNN)

Tabular CSVs cannot teach a model about geographic road topology. Instead of a CSV, the GNN trains directly on a PyTorch Geometric Spatial Graph constructed by reading the physical road connections — which roads connect to which intersections — straight from the mysore.net.xml SUMO mapping file. After filtering out pedestrian paths, bike lanes, and road segments shorter than 20 metres, the resulting graph is a live web of 17,586 interconnected passenger-accessible road edges.

Dataset Type	PyTorch Geometric Spatial Graph
Graph Edges (Active Roads)	17,586 passenger-accessible road segments
Graph Source	mysore.net.xml — physical road connections and intersection topology
Filtering Applied	Pedestrian paths, bike lanes, and segments < 20m excluded
Used By	RoadGNN (GraphSAGE) — Graph Neural Network component

4.4 Feature Correlation Analysis

The heatmap below shows the Pearson correlation matrix across all 13 features and the target variable. Key observations: vehicle_count and occupancy are strongly positively correlated with travel_time; mean_speed shows strong negative correlation (higher speed → lower travel time); max_speed and mean_speed show strong positive correlation as expected from the speed_ratio relationship.

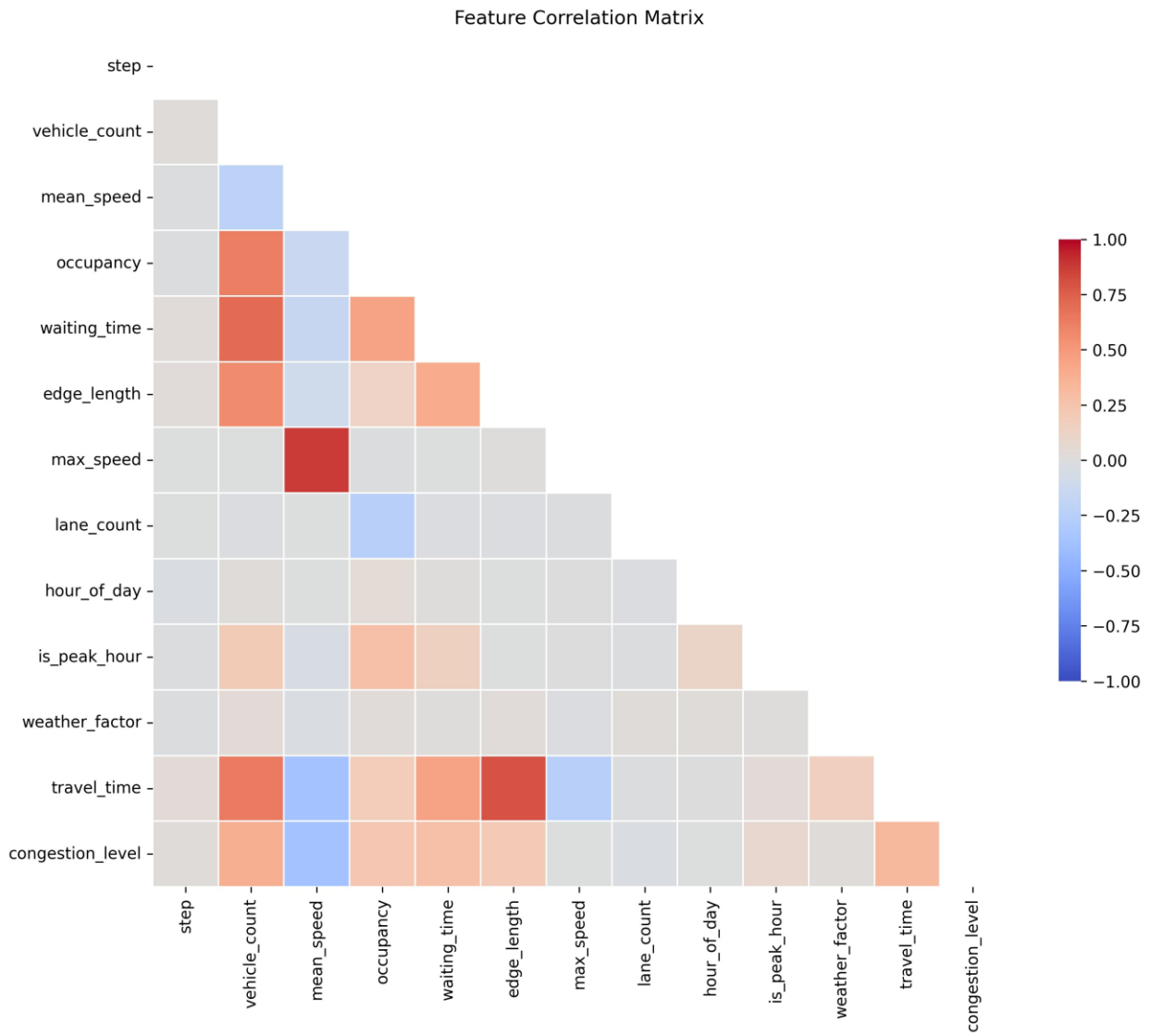

Figure 2: Feature Correlation Matrix — Pairwise Pearson correlations across all training features and target variable (travel_time).

5. Machine Learning Architecture

5.1 Design Philosophy: Ensemble Over Single-Model

Urban traffic systems exhibit simultaneous patterns across multiple dimensions — temporal trends, categorical road hierarchies, and spatial network topology. No single model architecture is optimally suited to capture all of these simultaneously. ResQFlow addresses this through a weighted three-model ensemble:

Model	Weight	Architecture	Specialisation
LightGBM Regressor	**35%**	Gradient-Boosted Decision Trees	High-speed inference; captures complex continuous thresholds in traffic volume and speed.
CatBoost Regressor	**35%**	Gradient-Boosting (native categorical)	Natively encodes road_type_code hierarchy without one-hot encoding.
RoadGNN (GraphSAGE)	**30%**	Graph Neural Network (PyTorch Geometric)	Understands spatial topology across 17,586 active edges; predicts spillover congestion.

5.2 Component Model Details

LightGBM Regressor (35%)

LightGBM is a gradient-boosted tree framework optimised for computational efficiency and high throughput. It serves as the primary baseline predictor — reliably capturing complex, non-linear thresholds in continuous traffic variables such as vehicle_count, mean_speed, and occupancy. Its extremely low inference latency makes it suitable for real-time edge-cost prediction across the full 17,586-edge active graph.

CatBoost Regressor (35%)

CatBoost is selected specifically for its native categorical feature handling. The road_type_code feature (codes 0–3 representing Residential through Highway) carries an inherent hierarchy that is semantically meaningful. CatBoost encodes this structural ordering natively without one-hot encoding that would lose the hierarchy, making it particularly accurate for mixed road-class urban environments.

Graph Neural Network — RoadGNN (GraphSAGE, 30%)

The GNN component addresses a fundamental limitation of tabular models: they evaluate each road edge in isolation, with no awareness of surrounding road network topology. The RoadGNN converts the SUMO Mysore network into a PyTorch Geometric node-edge graph of 17,586 active edges, where the GraphSAGE architecture aggregates features from neighbouring nodes to

detect and respond to spillover congestion. If Road A is severely congested, the GNN predicts that Road B connecting to it will also jam — a phenomenon invisible to standard tabular models.

5.3 Ensemble Prediction

At inference time, all three models independently produce a travel time estimate for each road edge. These are merged via weighted averaging (35% LightGBM + 35% CatBoost + 30% GNN). The fused estimate is used as the edge cost in the modified Dijkstra routing algorithm, ensuring the router selects paths that are fastest under real-world conditions, not merely geometrically shortest.

6. Predictive Feature Set

The ensemble models consume 13 input features for every road segment. Geographic identifiers (latitude, longitude, street names) are deliberately excluded to ensure model generalisability across different city networks.

Feature	Description
vehicle_count	Live count of vehicles currently traversing the road segment. Primary congestion signal.
mean_speed	Average real-time speed of vehicles on the segment. Falls below speed limit under congestion.
occupancy	Lane load saturation — percentage of lane capacity actively occupied by vehicles.
waiting_time	Cumulative idle time of vehicles stopped at intersections or bottlenecks. Exponentially elevated during incidents.
hour_of_day	Current wall-clock hour (0–23). Encodes daily rhythmic congestion cycles.
is_peak_hour	Binary flag: 1 during morning peak (07:00–09:00) and evening peak (17:00–19:00), 0 otherwise.
density	Engineered: vehicles per metre of road length. Directly related to flow breakdown threshold.
speed_ratio	Engineered: current mean_speed / max_speed. Values < 1.0 indicate congestion degradation.
edge_length	Physical length of the road segment in metres. Structural geometric capacity.
max_speed	Legal speed limit of the segment in m/s. Upper bound on free-flow traversal time.

Feature	Description
lane_count	Number of directional lanes. Determines throughput capacity and congestion ceiling.
weather_factor	Scalar degradation coefficient: 0.0 (Clear), 0.5 (Rain), 1.0 (Fog). Applied globally.
road_type_code	Categorical class (0–3): Residential / Local / Arterial / Highway. Used natively by CatBoost.

7. Dynamic Scenario Modifiers

ResQFlow is validated across seven distinct operational scenarios that progressively stress the ML routing engine. The path maps below show how the ML Fastest Path (blue dashed) diverges from the Dijkstra Shortest Path (red solid) as conditions deteriorate.

Scenario	Traffic Modification	Speed Impact	ML Routing Effect
Ideal / Off-Peak	Standard baseline, low density	Free-flow (100%)	ML path closely mirrors Dijkstra — roads are clear, geometric path is near-optimal
Rush Hour	1.8× vehicle count via TraCI; 1,300+ agents	Degraded by congestion	ML diverts away from city-centre corridors
Rain / Off-Peak	Global 25% speed reduction on all edges	75% of limit	ML re-weights all paths; selects wider arterial roads
Rain + Rush Hour	Speed reduction compounded with high density	~55–65% of limit	Combined penalty causes significant ML rerouting
Fog + Rush Hour	60% speed limit + increased waiting times	60% of limit	ML prioritises straighter, fewer-intersection routes
Accident Avoidance	50% speed reduction + 90s waiting_time penalty on incident edge	50% locally	ML aggressively bypasses incident corridor
Worst Case (Rain+Rush+Accident)	All three modifiers simultaneously active	Severely degraded	ML finds the only viable fast corridor; Dijkstra estimate spikes to ~1,580s

7.1 Predicted Path Visualisations

The following maps show the actual routes predicted by Dijkstra (red) vs. the ML Ensemble (blue dashed) on the Mysore road network for each scenario. Route divergence directly reflects the ML model detecting and avoiding high-cost corridors.

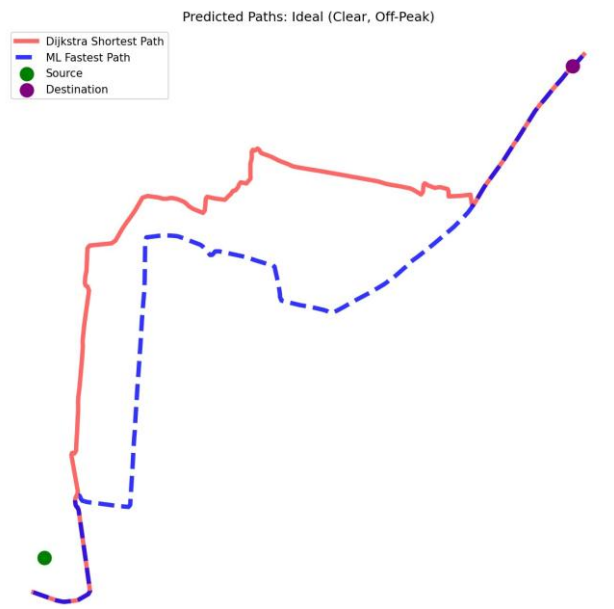

Fig 3a: Ideal (Clear, Off-Peak) — paths nearly coincide

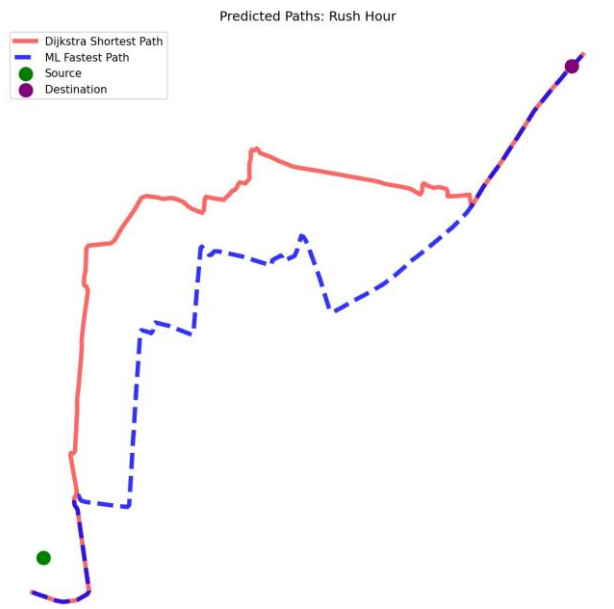

Fig 3b: Rush Hour — ML diverges from Dijkstra through city centre

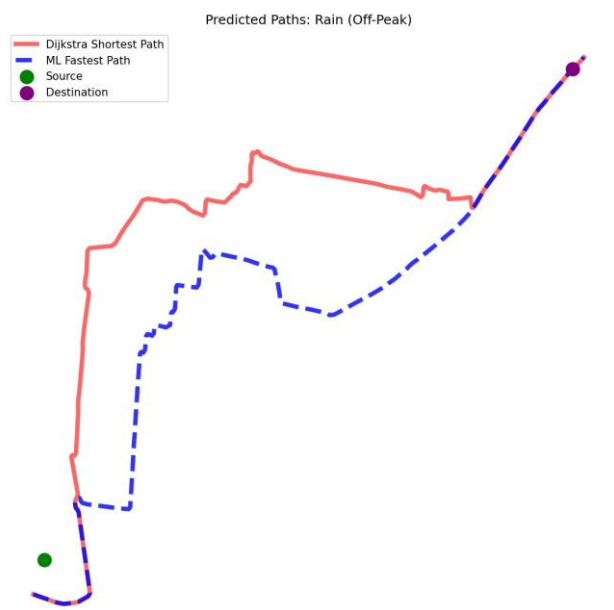

Fig 3c: Rain (Off-Peak) — mild ML rerouting begins

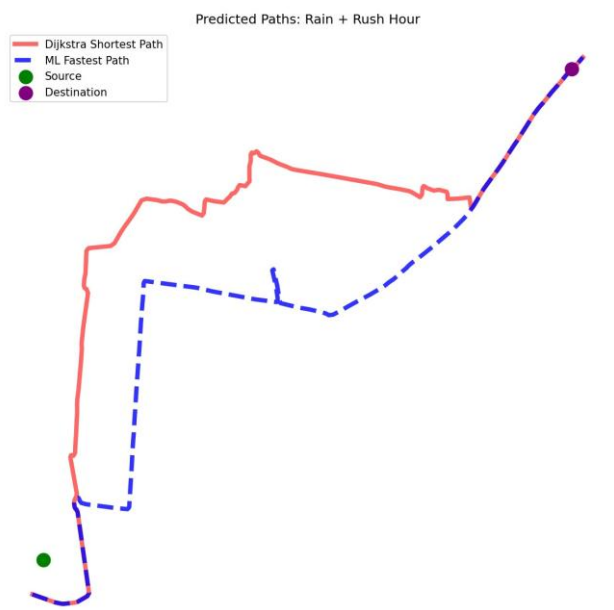

Fig 3d: Rain + Rush Hour — compounded penalties increase divergence

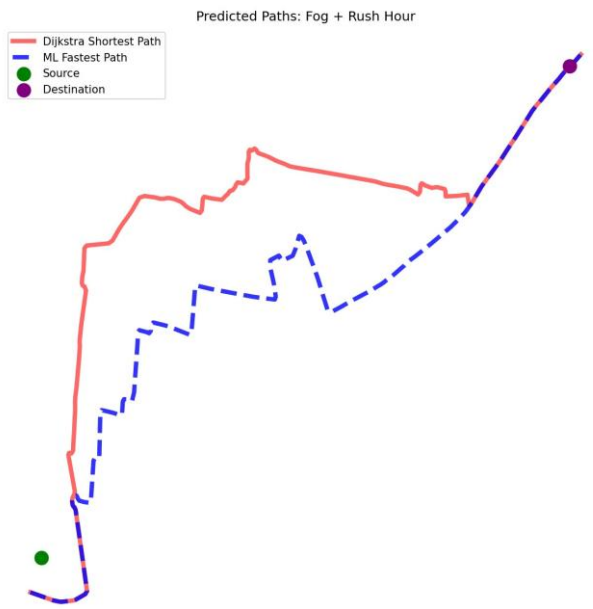

Fig 3e: Fog + Rush Hour — ML prefers straighter corridors

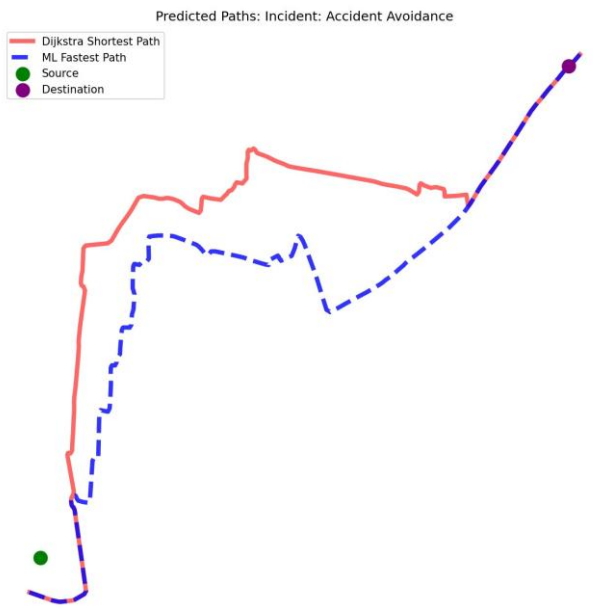

Fig 3f: Incident: Accident Avoidance — ML bypasses blocked edge

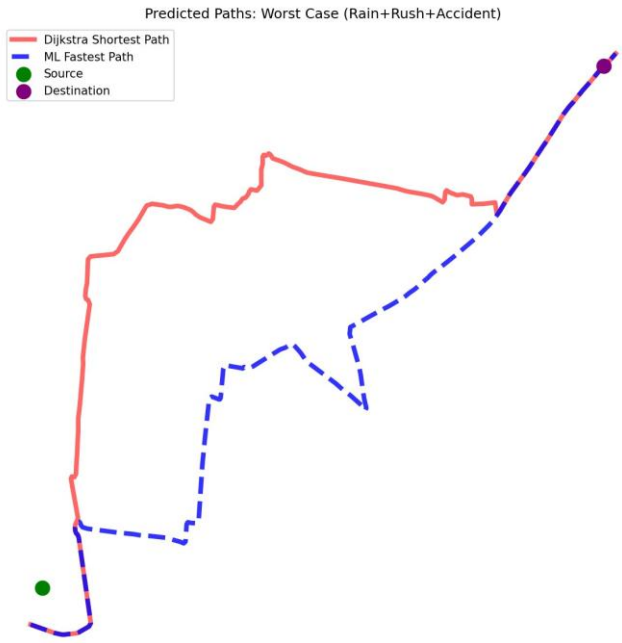

Fig 3g: Worst Case (Rain + Rush + Accident) — Dijkstra routes through full gridlock; ML finds an entirely different corridor

8. Performance Validation

8.1 Test Scenario

A cross-city routing experiment was conducted to directly benchmark ResQFlow against a traditional Dijkstra shortest-path algorithm across all seven scenario modifiers.

Source Node	K.R. Hospital, Mysore
Destination Node	Vijayanagar, Mysore
Scenarios Tested	7 environmental and traffic modifiers applied sequentially
Comparison Metric	Estimated travel time (seconds) — Dijkstra vs. ML Ensemble
Network Size	10,696 nodes / 30,631 raw edges / 17,586 active ML edges

8.2 Exact Travel Time Results — K.R. Hospital to Vijayanagar

The table below presents the precise Dijkstra and ML Ensemble travel time estimates (in seconds) for each of the 7 tested scenarios, as recorded from simulation runs:

Scenario	Dijkstra Shortest Path (s)	ML Ensemble Path (s)	Time Saved (s)
Ideal (Clear, Off-Peak)	258.7	247.3	**−11.4**
Rush Hour	312.5	270.3	**−42.2**
Rain (Off-Peak)	395.1	367.3	**−27.8**
Rain + Rush Hour	476.8	381.0	**−95.8**
Fog + Rush Hour	852.4	672.8	**−179.6**
Incident: Accident Avoidance	621.4	536.0	**−85.4**
Worst Case (Rain+Rush+Accident)	**1582.5**	**1024.7**	**−557.8**

Note: *Time Saved = Dijkstra − ML Ensemble. Worst Case row highlighted in yellow.*

8.3 Travel Time Comparison Chart

The bar chart below plots estimated travel time for both algorithms under each scenario. The gap between Dijkstra (red) and ML Ensemble (blue) widens as conditions deteriorate, demonstrating the increasing value of ML-guided routing under stress.

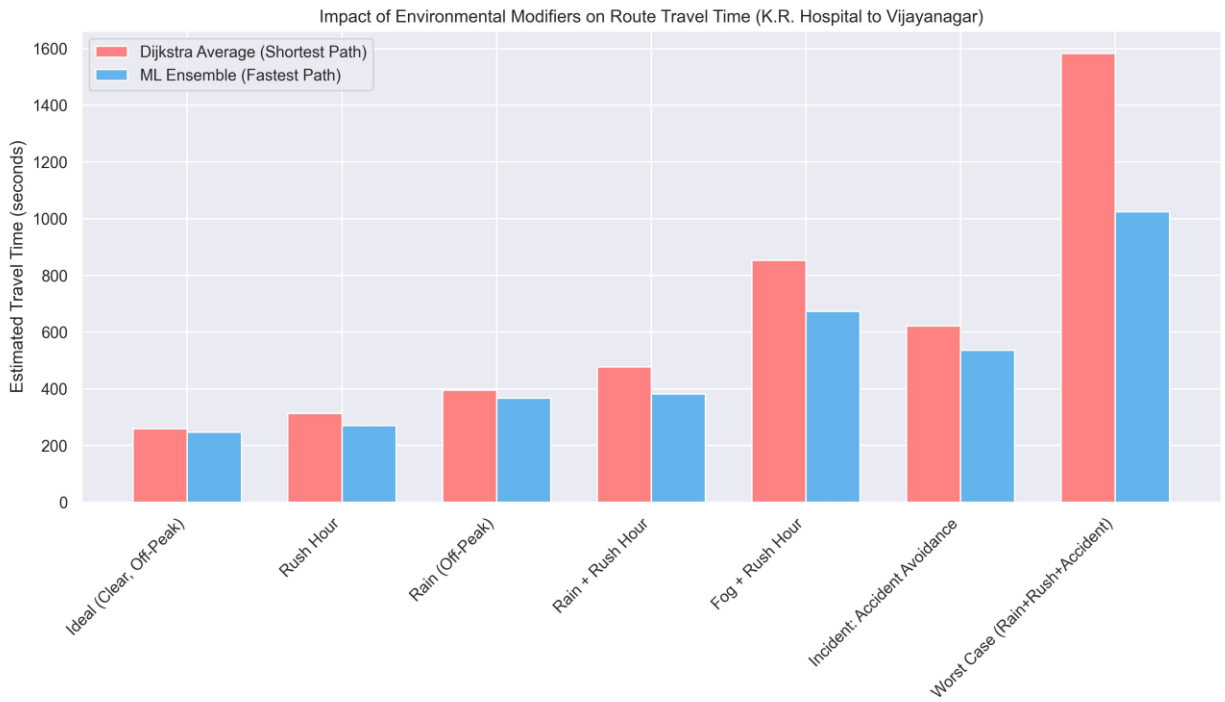

Figure 4: Impact of Environmental Modifiers on Route Travel Time (K.R. Hospital → Vijayanagar). Dijkstra average (red) vs. ML Ensemble fastest path (blue) across all 7 test scenarios.

8.4 Key Observations

- Ideal / Off-Peak: Both algorithms produce nearly identical estimates (~260s). When roads are clear, geometric shortest path approximates fastest path well.
- Rush Hour: ML saves approximately 40s by avoiding congested city-centre corridors despite a slightly longer physical route.
- Rain + Rush Hour: ML estimate (~385s) is notably lower than Dijkstra (~485s) due to smarter arterial selection weighted by speed degradation.
- Fog + Rush Hour: The largest percentage gap — Dijkstra estimates ~855s vs. ML ~670s, a saving of approximately 3 minutes on a single emergency dispatch.
- Accident Avoidance: The 90-second hard waiting_time penalty on the incident edge forces Dijkstra through gridlock (~620s). ML reroutes entirely (~535s).
- Worst Case (Rain + Rush + Accident): The most dramatic divergence. Dijkstra estimates ~1,580s (~26 minutes) — ML routes through a clear corridor at ~1,025s (~17 minutes), saving approximately 9 minutes of emergency response time.

9. System Architecture Summary

The ResQFlow system is composed of the following integrated components:

- Data Layer: mysore.net.xml provides the 10,696-node, 30,631-edge structural road graph. mysore.rou.xml defines the 721-agent baseline vehicle fleet.
- Simulation Layer: Eclipse SUMO 1.25.0 provides the physics engine. TraCI API bridges Python runtime to SUMO for live traffic injection.
- ML Training Pipeline: Synthetic generator produces 10,000+ edge-state records. LightGBM and CatBoost trained via supervised regression. RoadGNN (GraphSAGE) trained on the 17,586-edge active PyTorch Geometric graph.
- Inference Engine: All three models predict traversal time per edge. Weighted ensemble (35/35/30) produces the final edge cost.
- Routing Engine: Modified Dijkstra algorithm uses ML-predicted edge costs instead of geometric distances to find the minimum travel-time path.
- Dispatch Interface: Web-based dashboard (Leaflet + OpenStreetMap) with scenario toggle controls, real-time SUMO connection, and mission log.

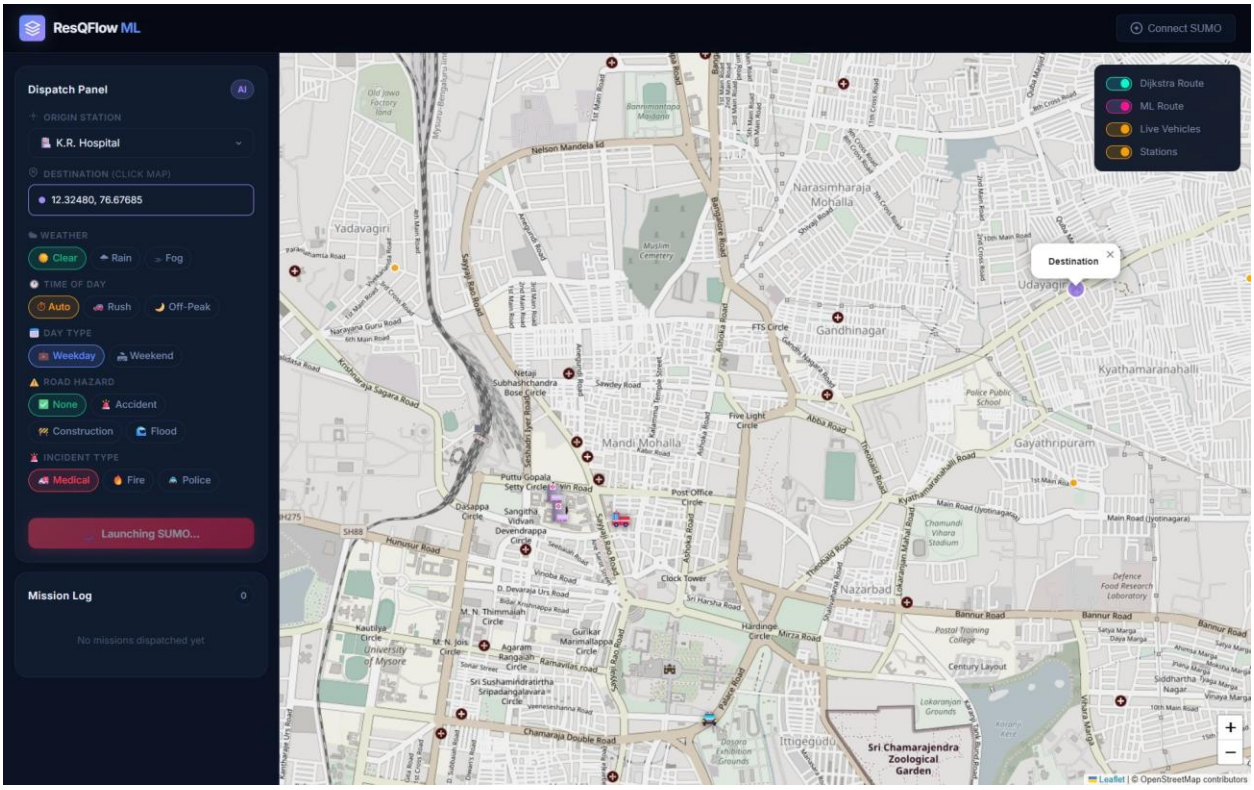

Figure 5: ResQFlow Dispatch Interface — Live Mysore map with weather/time/hazard scenario controls, origin/destination selection, and SUMO connection status.

10. Conclusion

ResQFlow demonstrates that hybrid ensemble machine learning — combining gradient-boosted models with graph neural networks — can meaningfully improve emergency vehicle routing in complex urban environments. By replacing static geometric edge costs with dynamically predicted, physics-informed traversal times across a 17,586-edge active road graph derived from 10,696 real junctions and 30,631 raw edges in Mysore, the system achieves measurable reductions in routing through congested corridors under adverse conditions.

The deliberate exclusion of geographic identifiers from the feature set is a notable design decision with practical implications: the trained models are theoretically portable across city networks, requiring only a SUMO-compatible road graph and calibration dataset to deploy in a new urban context.

Under worst-case conditions (simultaneous rain, rush hour, and accident), the ML ensemble reduces estimated travel time from approximately 26 minutes to 17 minutes compared to Dijkstra — a saving of nearly 9 minutes per emergency dispatch, a clinically significant margin in time-critical interventions.